

Nifty100 Financial Intelligence Platform

Analyst Guide & Project Documentation

Version 1.0

Table of Contents

1. Project Overview
2. System Architecture
3. Folder Structure
4. Database Schema
5. Data Pipeline
6. Financial Ratio Engine
7. Screener Engine
8. Peer Comparison Engine
9. Streamlit Dashboard
10. Valuation Module
11. Cash Flow Intelligence
12. NLP Pros & Cons Engine
13. FastAPI Backend
14. Reports Generation
15. Configuration Files
16. API Usage Examples
17. KPI Definitions
18. Installation Guide
19. Dashboard Usage
20. Troubleshooting
21. Future Enhancements

1. Project Overview

The Nifty100 Financial Intelligence Platform is an automated analytics ecosystem that tracks, analyzes, and visualizes the fundamental health of the top 100 Indian companies. It consolidates data from multiple sources, generates actionable insights through clustering and NLP models, and presents the findings through both a Streamlit UI and a FastAPI backend.

2. System Architecture

The platform employs a modular architecture: - ETL Layer: Ingests raw CSVs, normalizes headers, calculates metrics, and populates SQLite. - Analytics Layer: Computes 30+ KPIs, sector averages, scoring mechanisms, K-Means clustering, and Cashflow analytics. - Backend API: A RESTful FastAPI server exposing real-time insights. - Frontend: An interactive Streamlit dashboard consisting of 8 analytical views. - Reporting: Automated PDF generation via ReportLab.

3. Folder Structure

Root/ - src/ : Core logic (api, analytics, dashboard, etl, nlp, reports) - data/ : Database (nifty100.db) and raw/processed CSVs - docs/ : Documentation (analyst_guide.pdf) - output/ : CSVs, Excel summaries, clustering data - reports/ : Generated PDFs (Tearsheets, Sector, Portfolio, QA)

4. Database Schema

The SQLite database (`nifty100.db`) consists of tightly linked tables via foreign keys: - `companies`: Master ticker list and metadata. - `sectors`: Broad and sub-sector classifications. - `financial\_ratios`: Computed yearly KPIs (ROE, ROCE, FCF, etc.). - `market\_cap`, `profitandloss`, `cashflow`, `balancesheet`. - `prosandcons`: Inferred textual highlights.

5. Data Pipeline

The ETL pipeline normalizes semi-structured flat files. `database\_loader.py` merges disjointed datasets into standardized DataFrames, aligns years chronologically (filling gaps up to 10 years), handles missing data (NaN dropping/interpolation), and loads them securely into the SQLite relational model.

6. Financial Ratio Engine

Computes key ratios including: - Profitability: ROE, ROCE, NPM, OPM. - Solvency: Debt-to-Equity, Interest Coverage Ratio (ICR). - Growth: 3, 5, and 10-year CAGRs (Revenue, PAT, EPS).

7. Screener Engine

A flexible querying system that ranks companies by a Composite Quality Score (combining ROE percentiles, valuation discounts, and earnings growth). Supports presets like 'Value Traps', 'High Growth', and 'Distressed'.

8. Peer Comparison Engine

Conducts intra-sector comparisons by normalizing metrics across companies within the same `broad\_sector`. Visualized via Radar charts comparing individual tickers against Sector Averages.

9. Streamlit Dashboard

Features 8 distinct screens: 1. Home: Market overview. 2. Profile: Detailed company teardown. 3. Screener: Filterable grid view. 4. Peers: Radar charts & comparative tables. 5. Trends: Time-series Line charts. 6. Sectors: Aggregate KPI visualizations. 7. Cashflow: Cash-burn/Returns classification. 8. Reports: PDF download hub.

10. Valuation Module

Tracks enterprise value multipliers. Specifically computes P/E ratio, P/B ratio, and EV/EBITDA. Flags companies trading below intrinsic historical averages.

11. Cash Flow Intelligence

Calculates Free Cash Flow (CFO - CapEx), CFO Quality Score, and Capex Intensity. Categorizes companies into one of 8 structural patterns (e.g., 'Cash Cow', 'Aggressive Expansion', 'Distressed').

12. NLP Pros & Cons Engine

Utilizes regex-based rules to ingest raw financial parameters and output human-readable insights. Identifies high growth trajectories as 'PROs' and over-leveraged balances as 'CONS' with assigned confidence percentages.

13. FastAPI Backend

Production-grade backend exposing 16 endpoints. Leverages Pydantic for validation, CORS for integration, and generator functions for per-request database pooling.

14. Reports Generation

Engineered to produce dynamic multi-page PDFs using ReportLab. Generates Individual Tearsheets, Sector-wide aggregates, and Alphabetically-sorted Portfolio summaries with visual trend arrows (↑, ↓, →).

15. Configuration Files

All environmental paths are dynamically resolved via ``Path(__file__)``. Requirements are locked in ``requirements.txt``. Built against Python 3.12+.

16. API Usage Examples

- ``GET /companies``: Lists all tickers. - ``GET /ratios/RELIANCE``: Retrieves 10-yr history. - ``POST /recommend``: Body ``{'risk_profile': 'Low'}`` returns top 3 Mutual Funds.

17. KPI Definitions

- **ROE**: $\text{Net Income} / \text{Shareholder Equity}$ - **CAGR**: $((\text{End Value} / \text{Start Value})^{(1/\text{Years})})$
- 1 - **CFO Quality**: $\text{Operating Cash Flow} / \text{Net Profit}$ - **FCF Conversion**: $\text{Free Cash Flow} / \text{Net Profit}$

18. Installation Guide

1. Clone repository. 2. ``python -m venv .venv`` 3. ``.\.venv\Scripts\activate`` 4. ``pip install -r requirements.txt``

19. Dashboard Usage

Run ``streamlit run src/dashboard/app.py``. Use the sidebar to navigate between modules. Ensure the SQLite database is populated first using ``python src/etl/database_loader.py``.

20. Troubleshooting

If the dashboard fails to load missing columns, ensure the Sprint 6 analytics modules (``clustering.py``, ``cashflow_kpis.py``) were executed to populate the ``output/`` artifacts.

21. Future Enhancements

Future roadmap includes: - Real-time API integration for stock quotes. - NLP-based sentiment analysis on earnings call transcripts. - Deep Learning models for bankruptcy prediction.